Advanced RNN Topics

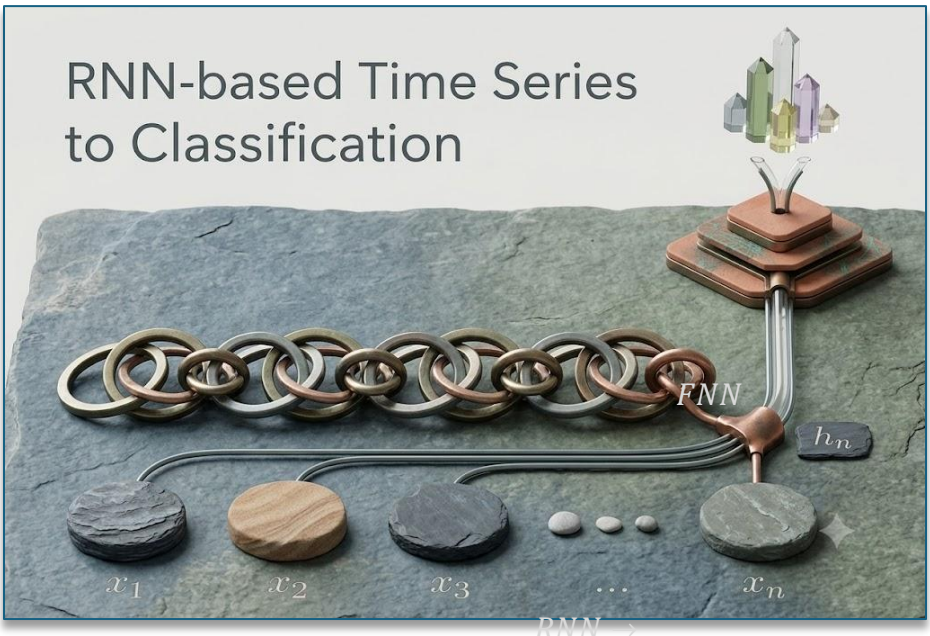
Overview

- **enumerate advanced RNN architectures including GRUs and LSTMs for handling long-term dependencies**
- **implement sampling and beam search strategies for generating sequences**

Async

Advanced Topics with RNNs

- using RNNs for classification
  - take the last token's hidden representation (h_n) as the representation of the sequence
  - loss computed at the FFN's softmax layer



Gemini 3.5

  - errors flow back to RNN (end to end training)
- alternative:
  - element-wise mean or max over states
  - example: $h_{seq} := \frac{1}{n}\sum h_i$

- RNN Training
  - use a text corpus – make the RNN model predict the next word at each time step t
    - RNN does not predict the word directly
    - at each time step t, it predicts a probability distribution over vocab: $\hat{y}_t$
    - next word is sampled from this probability distribution: $\hat{y}_t$

$$h_t = \sigma(W_1 h_{t-1} + W_2 x + b)$$

$$\hat{y}_t = \lambda(W_3 h_t)$$

  - objective function is to minimize the error in predicting the next word using *cross-entropy* as the loss function

- Cross-Entropy Loss for Sequence Modeling
  - measures the difference between a predicted probability distribution and the correct distribution
    - correct distribution (over full vocabulary): y_t
    - predicted distribution (over full vocabulary): $\hat{y}_t$

$$L_{CE} = -\sum_{w \in V} y_t[w] \log \hat{y}_t[w]$$

    - probability of a word w under a distribution:

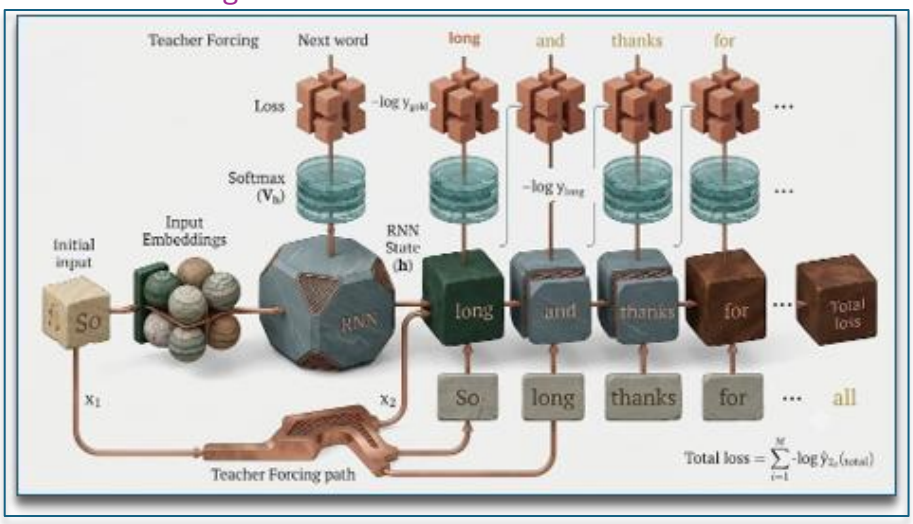
$$y_t = y_t[w]$$

  - in language modeling
    - we know the next word (y_t) is a one-hot vector with $dim = |V|$ where the entry for the next word is 1 and the rest are 0

- cross-entropy loss at time t is:

$$L_{CE}(\hat{y}_t, y_t) = -\log \hat{y}_t\,[w_{t+1}]$$

- Teacher Forcing



Gemini 3.5

  - at train time, feed the initial input word at t irrespective of what the model predicted at $t-1$
  - embeddings e_t can be obtained by multiplying input (one-hot) vector of x_t with matrix $E_{d_h \times |\tilde{V}|}$, where $\tilde{V} = vocabulary$
  - E is the input embedding matrix
    - columns of E are the embeddings for words in $\tilde{V}$

Gradient Computation

- Gradient Computation with RNNs
  - RNNs can still be trained with backpropagation
  - gradient involved repeated multiplications of Ws

$$h_t = \tanh(Ux_t + Wh_{t-1})$$

$$z_t = Ux_t + Wh_{t-1}$$

$$h_t = \tanh(z_t)$$

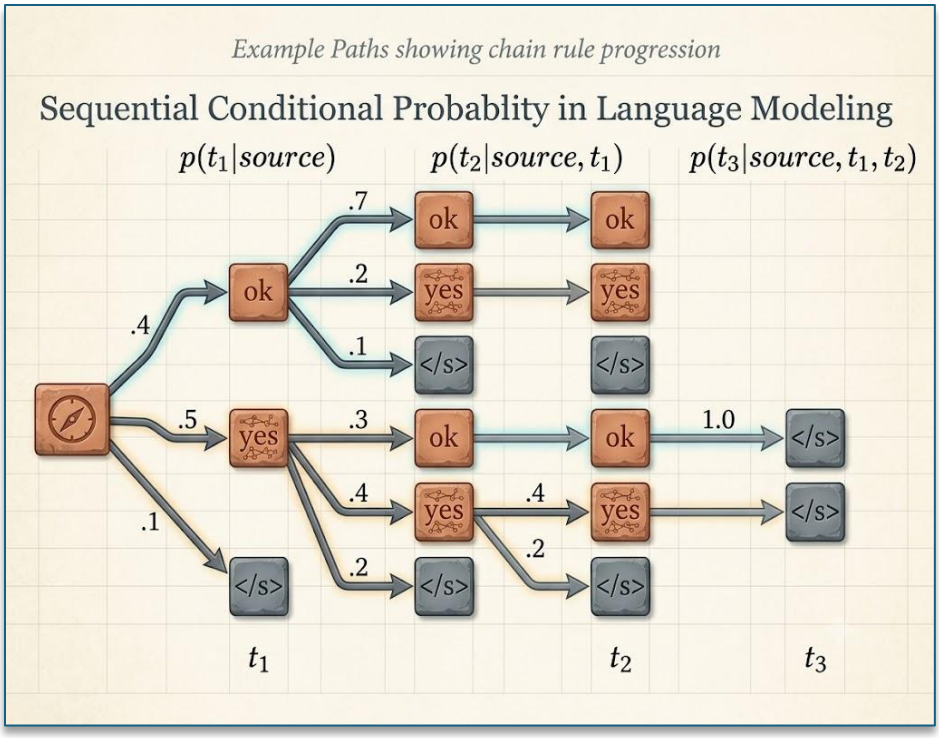
$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k}\frac{\partial h_k}{\partial W}$$

$$\frac{\partial h_k}{\partial W} = \frac{\partial h_k}{\partial z_k}\frac{\partial z_k}{\partial W}$$

$$\frac{\partial z_k}{\partial W} = h_{k-1} + W\frac{\partial h_{k-1}}{\partial W}$$

	○ the issue with W being continually propagated through this chain is, depending on the state of W, the gradient will likely either vanish or explode • solutions: ○ special activations ▪ gradient of ReLU activation avoids this ○ normalization techniques ▪ batch and layer normalization to normalize the activations of a network ○ gradient clipping ▪ clipping to a max value avoids gradient explosion
Greedy and Beam Search	• greedy decoding: ○ at any time step, generate the most likely word: $\hat{w}_t = argmax_{w \in V} P(w \mid w_{<t})$ ○ locally optimal ○ leads to extremely predictable results ▪ output is generic and repetitive ○ not commonly used in practice • beam search is a good alternative ○ common in very constrained tasks like machine translation • alternatives commonly used in LMs ○ top-k sampling ○ top-p/nucleus sampling ○ temperature sampling

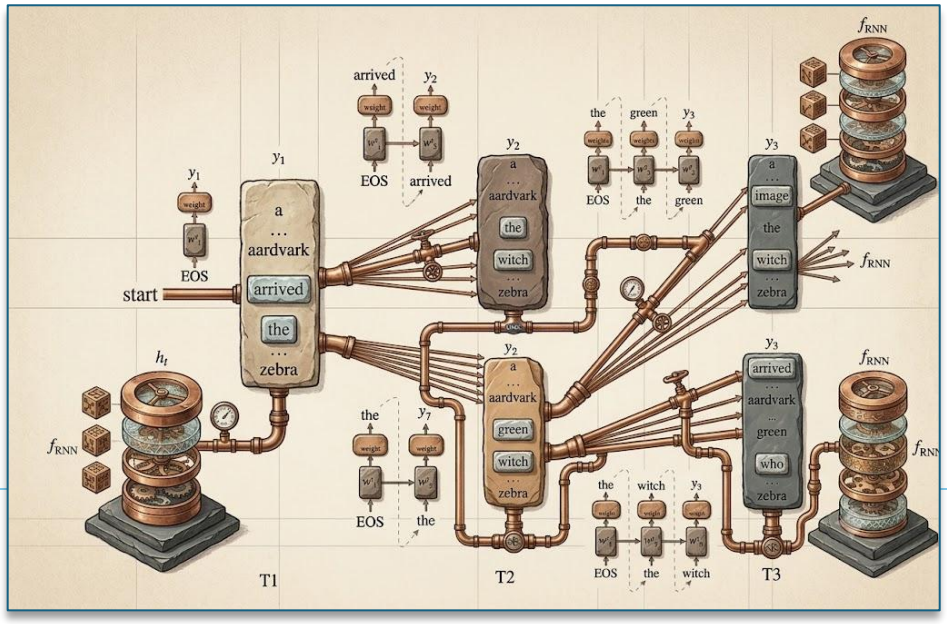
- greedy search tree
  - representation of the search space for the output
  - branches represent possible actions



Gemini 3.5

  - the greedy output (taking argmax)
    - $yes, yes, </s>$
    - $probability = 0.5 * 0.4 * 1.0 = 0.2$
  - highest probability output
    - $ok, ok, </s>$
    - $probability = 0.4 * 0.7 * 1.0 = 0.28$
  - greedy search may not uncover the global optimal output
    - to obtain, execute exhaustive search of all possible sequences
    - for a vocabulary of size V and an output of length T
      - you must explore V_T sequences

- Beam Search
  - keep track of the K best possibilities at each step
    - K would be called the beam width
      - generally ~5 or 10
  - at $i = 1$
    - given x (source text), obtain probability distribution over entire vocabulary
    - keep the top K (highest probability) tokens
  - for $i = 2$ onwards
    - obtain probability distributions over entire vocabulary given the K possibilities from $i - 1$
      - $K \times V$ options
    - compute probability score for each of the $K \times V$ options
      - $P(y_i|x, y_1, y_2, \dots, y_{i-1})$
    - keep the top K highest scoring options out of the $K \times V$ options
      - s
  - result:
    - K paths, now pick the highest scoring among them



Gemini 3.5

Advanced Sampling Methods

- by only generating the highest probability words, you can end up with grammatical, but predictable and repetitive sentences
- sometimes picking the 2nd, 3rd, or 4th word might be more interesting
- two important factors
  - quality
    - highest probability words
  - diversity
    - high, not highest probability words

- top-k sampling
  - simple generalization of greedy decoding
    - choose in advance a number of words k
    - truncate the distribution to the top k most likely words
    - renormalize to produce a legitimate probability distribution
    - randomly sample from within these k words according to their renormalized probabilities
  - when $k = 1$:
    - top-k sampling = greedy decoding
  - $k > 1$:
    - select a word which is not the most probable, but still probable enough
    - results in generating more diverse but still high-quality text

- top-p / nucleus sampling
  - for top-k, k is fixed
    - for some contexts, original probability distribution can be flat
    - top-k words can cover most of the original probability mass
    - for other contexts, original probability distribution can be flat
    - top-k words will convert only a small part of the original probability mass
  - solution
    - use percent instead of absolute k value

$$\sum_{w \in V^p} P(w|w_{<t}) \geq p$$

- temperature sampling
  - don't truncate the distribution
  - reshape it instead
  - intuition from thermodynamics
    - a system at high temperature is flexible and can explore many possible states
    - while the system is at a lower temperature, it is likely to explore a subset of lower energy (better) states
  - low-temperature sampling
    - smoothly increase the probability of the most probable "common" words and decrease the probability of the "rare" words
  - reminder: with RNNs

	▪ hidden states from the last layer used to obtain scores (logits, V-dimensional) via a linear layer ▪ logits converted to a probability distribution using a softmax function ○ in temperature sampling ▪ divide the logits by a temperature, $\tau \in (0,1]$ before doing softmax $$y = softmax(u / \tau)$$
Summary	• greedy decoding ○ leads to extremely predictable results ▪ output is generic and repetitive • beam search is a good alternative • alternatives commonly used in LLMs ○ top-k sampling ○ top-p / nucleus sampling ○ temperature sampling